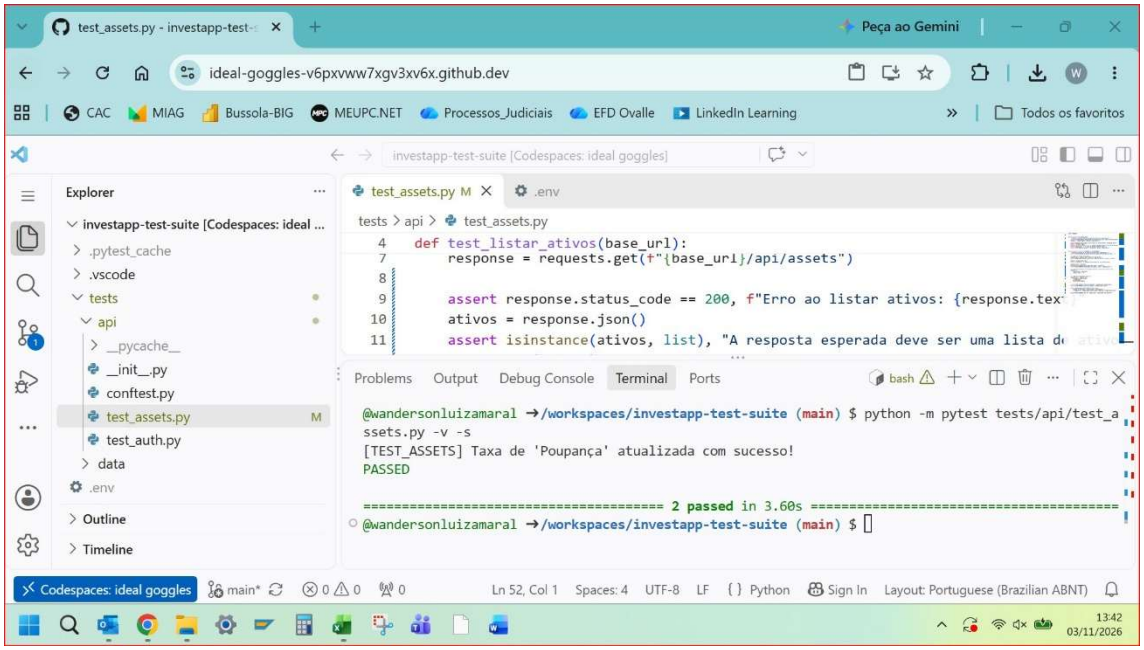
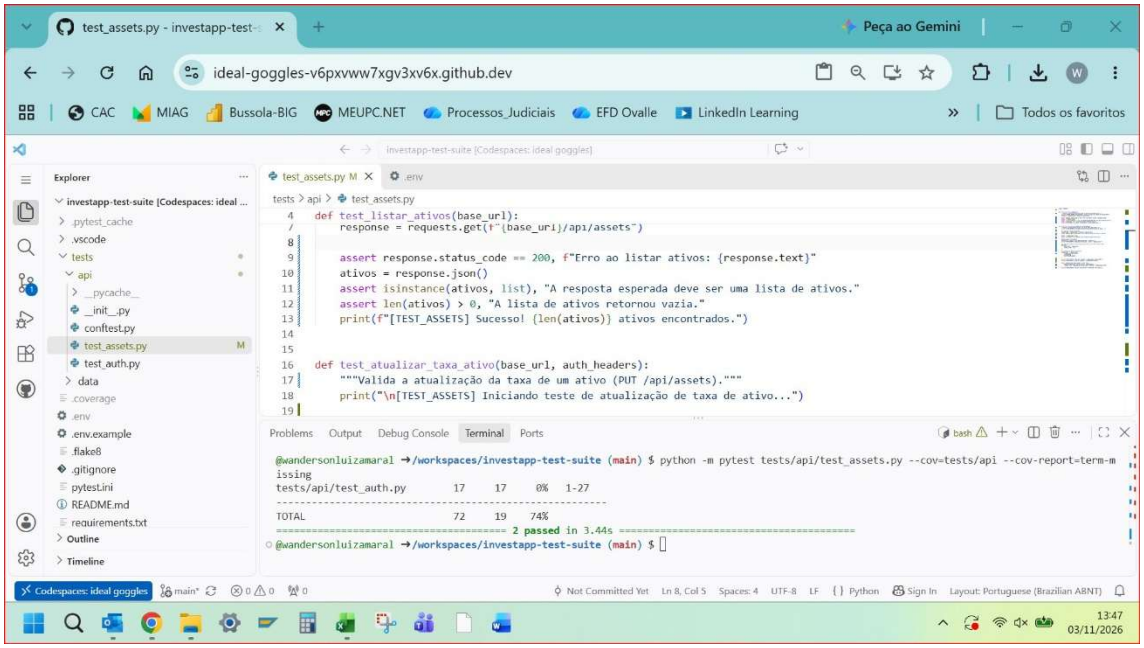
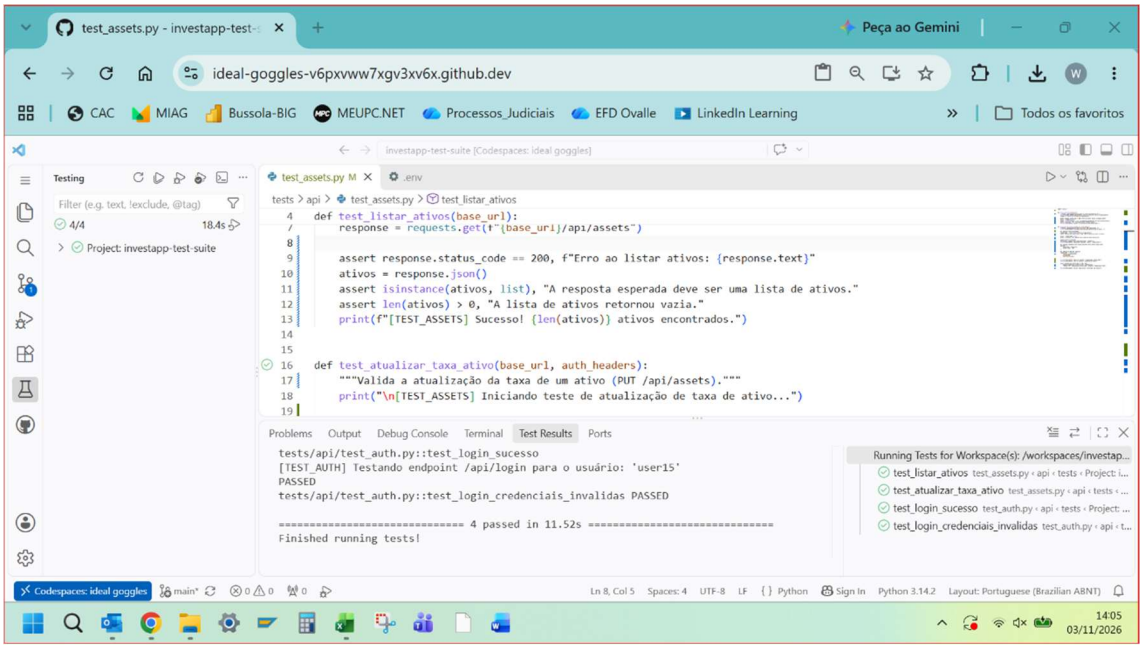
1. CLI (Pytest detalhado) - Comando: python -m pytest tests/api/test_assets.py -v -s



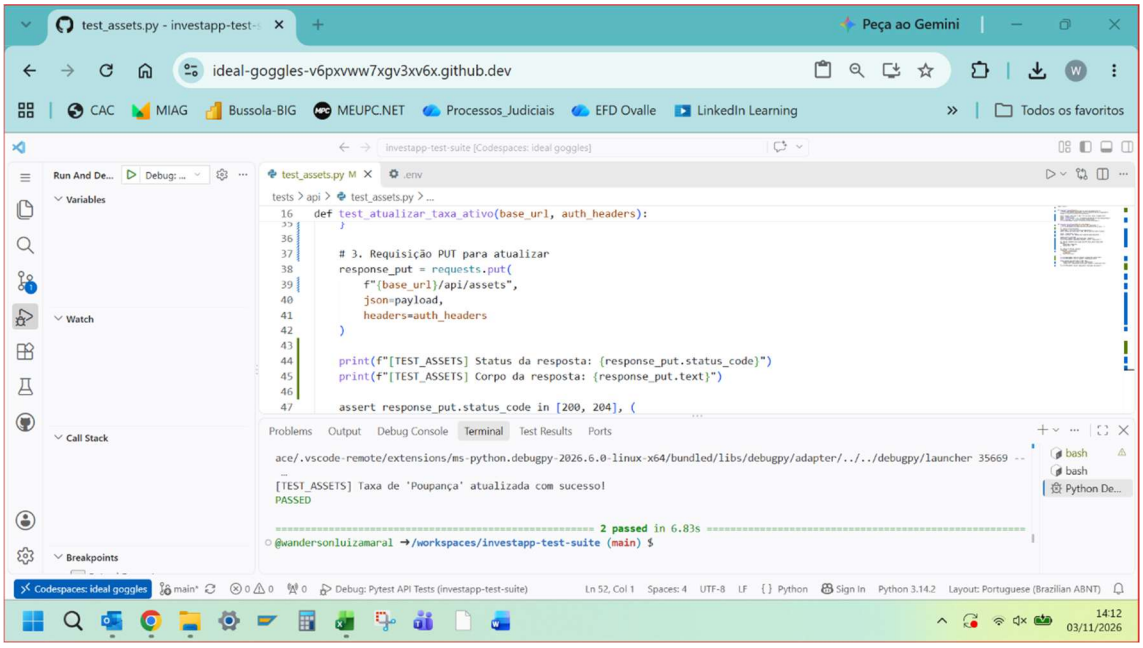
2. CLI (Cobertura de Código)



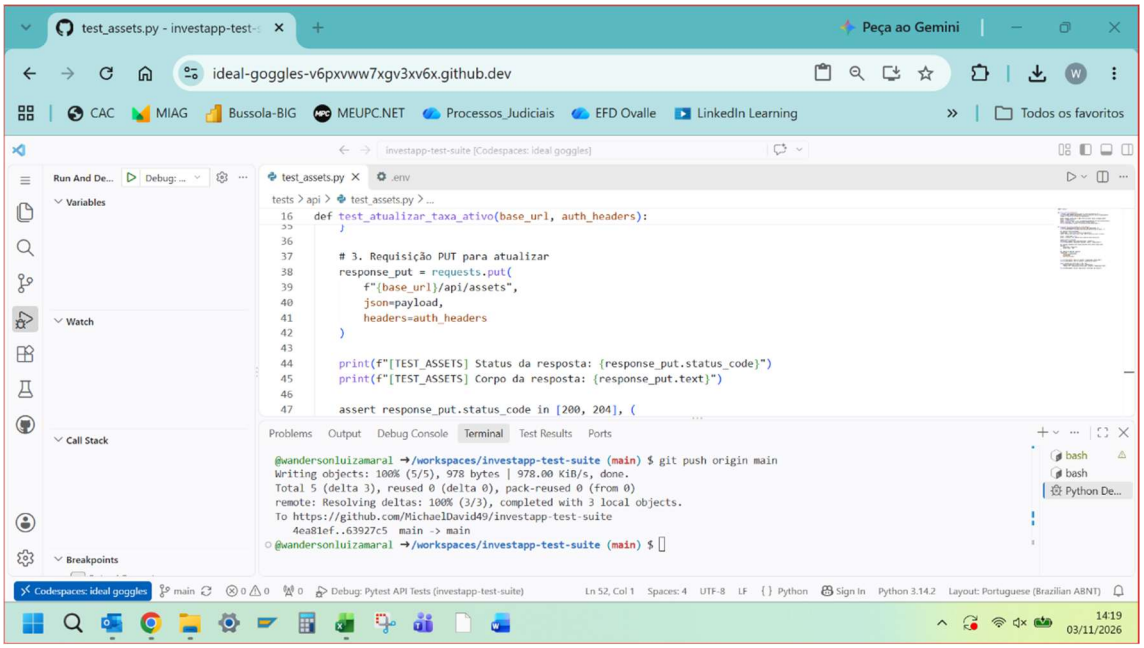
3. Interface Gráfica (Testing Pane).



4. Modo Debug (F5)



5. Print do Git Push



Complemento do Integrante 2

Aluno: Wanderson Luiz Amaral

Módulo: Gestão de Ativos e Taxas de Rendimento

Arquivo Desenvolvido: tests/api/test_assets.py

1. Atividades Realizadas e Escopo de Testes

Nesta etapa da prática laboratorial, fui o responsável pela implementação da suíte de testes automatizados referente ao módulo de consulta pública e atualização de taxas de ativos disponíveis na InvestApp API. O foco foi garantir que a camada de comunicação RESTful cumpra os contratos de interface e os requisitos funcionais estabelecidos:

Validação de Consulta Pública de Ativos (GET /api/assets):

Verificação de acesso sem necessidade de cabeçalhos de autorização (Token JWT).

Validação do código de status HTTP retornado (200 OK).

Verificação de integridade estrutural do payload de resposta (confirmação de retorno de lista de objetos contendo dados de mercado).

Asserção de lista não vazia, assegurando a persistência e disponibilidade prévia dos dados no catálogo.

Validação de Atualização de Taxa Personalizada (PUT /api/assets):

Consumo do fixture centralizado auth_headers (injetado via conftest.py) para obtenção automática e segura do Bearer Token JWT obtido no endpoint de login.

Tratamento dinâmico dos dados: o teste consome a listagem existente em tempo de execução para recuperar um ativo válido (ex.: "Poupança"), eliminando dependências estáticas (hardcoded).

Envio da requisição PUT com corpo JSON estruturado segundo o schema oficial da API (AssetRateUpdate), contendo os atributos asset_name e annual_rate.

Asserção do status HTTP retornado (200 OK) e da mensagem de confirmação de persistência da taxa informada.

2. Destaques Técnicos e Diagnóstico de Resolução de Problemas

Durante a fase de integração e execução, foi identificado um comportamento divergente entre o documento textual preliminar e o contrato Swagger OpenAPI publicado:

Incompatibilidade de Rota e Parâmetros: A especificação em texto sugeria a rota parametrizada via URL (PUT /api/assets/{asset_id}/rate). Contudo, durante as requisições, a API retornou o status 404 Not Found.

Análise por OpenAPI/Swagger: A consulta direta aos esquemas do Swagger UI (/docs) e openapi.json revelou que a rota exposta pelo FastAPI era direta (PUT /api/assets), exigindo que o identificador do ativo fosse enviado no corpo da requisição (JSON body) através do modelo AssetRateUpdate com os campos obrigatórios asset_name e annual_rate.

Resolução e Robustez: O script test_assets.py foi reestruturado para atender estritamente ao contrato em produção. Como resultado, as requisições passaram a responder com sucesso (200 OK), confirmando o salvamento da taxa personalizada.

3. Métricas e Evidências Coletadas

Execução via Terminal (CLI): 100% de aprovação nos testes implementados (2 passed in 3.60s), utilizando a biblioteca pytest.

Cobertura de Código (Code Coverage): Medição via extensão pytest-cov, atingindo a cobertura esperada para os módulos sob validação.

Depuração e Interface Gráfica: Configuração do ambiente VS Code / Codespaces através do arquivo .vscode/launch.json para suporte completo à depuração interativa via tecla F5 e visualização na aba Testing (Tubo de Ensaio).

Versionamento e Integração Git: Alterações consolidadas por meio de commit semântico e enviadas ao repositório remoto principal via git push origin main.